

Example, ExampleThreadGroup

Fabrice.Kordon@lip6.fr



Objectif de l'exemple

2

Jouer avec les threads et les groupes

- Leur affecter des priorités globales

Observer

- Peu de choses en fait



La classe UneThread

3

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

class UneThread implements Runnable {
    private String message;

    UneThread(String s){
        this.message=s;
    }

    private void trace (String m) {
        System.out.println ("UneThread " + Thread.currentThread().getName() + " : " + m);
    }

    public void run() {
        this.trace("je dis " + message);
        // Occuper le processeur
        int tmp = 0;
        for (int x = 0; x < 1000000 ; x++) {
            tmp = tmp + x / 2;
            Thread.yield();
        }
        this.trace("bye bye");
    }
}
```


La «classe principale»

4

```
public class ExempleThreadGroup1 {
    static int nbGroupes;
    static int nbThreads;

    public static void main(String[] args) {
        try {
            nbGroupes = Integer.parseInt(args[0]);
            nbThreads = Integer.parseInt(args[1]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, nombre de groupes, nombre de thread par groupe.");
            return;
        }
        System.out.println("Priorités : min =" + Thread.MIN_PRIORITY +
                           ", max =" + Thread.MAX_PRIORITY);
        ThreadGroup tg[] = new ThreadGroup[nbGroupes];
        for (int i = 1; i <= nbGroupes ; i++) {
            tg[i-1] = new ThreadGroup("groupe " + (i));
            if (i % 2 == 0) {
                tg[i-1].setMaxPriority(Thread.MIN_PRIORITY);
            } else {
                tg[i-1].setMaxPriority(Thread.MAX_PRIORITY);
            }
            for (int j = 1; j <= nbThreads; j++) {
                UneThread w = new UneThread("bonjour de " + i + "." + j);
                Thread t = new Thread(tg[i-1], w, "T-" + i + "." + j);
                t.setPriority(tg[i-1].getMaxPriority());
                t.start();
            }
        }
    }
}
```


La «classe principale»

4

```
public class ExempleThreadGroup1 {
    static int nbGroupes;
    static int nbThreads;

    public static void main(String[] args) {
        try {
            nbGroupes = Integer.parseInt(args[0]);
            nbThreads = Integer.parseInt(args[1]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println("Arguments requis, nombre de groupes, nombre de thread par groupe.");
            return;
        }
        System.out.println("Priorités : min =" + Thread.MIN_PRIORITY +
            ", max =" + Thread.MAX_PRIORITY);
        ThreadGroup tg[] = new ThreadGroup[nbGroupes];
        for (int i = 1; i <= nbGroupes ; i++) {
            tg[i-1] = new ThreadGroup("groupe " + (i));
            if (i % 2 == 0) {
                tg[i-1].setMaxPriority(Thread.MIN_PRIORITY);
            } else {
                tg[i-1].setMaxPriority(Thread.MAX_PRIORITY);
            }
            for (int j = 1; j <= nbThreads; j++) {
                UneThread w = new UneThread("bonjour de " + i + "." + j);
                Thread t = new Thread(tg[i-1], w, "T-" + i + "." + j);
                t.setPriority(tg[i-1].getMaxPriority());
                t.start();
            }
        }

        System.out.println("Qui est où?");
        for (int i = 0 ; i < nbGroupes; i++) {
            tg[i].list(); // afficher le contenu du groupe
        }
        System.out.println("Je n'ai plus rien à faire...");
    }
}
```


Quelques exécutions...

5

```
$ java ExempleThreadGroup1 3 2
Priorités : min =1, max =10
UnThread T-1.1 : je dis bonjour de 1.1
UnThread T-1.2 : je dis bonjour de 1.2
UnThread T-2.1 : je dis bonjour de 2.1
UnThread T-2.2 : je dis bonjour de 2.2
Qui est où?
UnThread T-3.1 : je dis bonjour de 3.1
UnThread T-3.2 : je dis bonjour de 3.2
java.lang.ThreadGroup[name=groupe 1,maxpri=10]
  Thread[T-1.1,10,groupe 1]
  Thread[T-1.2,10,groupe 1]
java.lang.ThreadGroup[name=groupe 2,maxpri=1]
  Thread[T-2.1,1,groupe 2]
  Thread[T-2.2,1,groupe 2]
java.lang.ThreadGroup[name=groupe 3,maxpri=10]
  Thread[T-3.1,10,groupe 3]
  Thread[T-3.2,10,groupe 3]
Je n'ai plus rien à faire...
UnThread T-1.2 : bye bye
UnThread T-2.2 : bye bye
UnThread T-3.1 : bye bye
UnThread T-3.2 : bye bye
UnThread T-1.1 : bye bye
UnThread T-2.1 : bye bye
```



La priorité a-t-elle un effet sur la terminaison des tâches?

Quelques exécutions...

5

```
$ java ExempleThreadGroup1 3 2
Priorités : min =1, max =10
UnThread T-1.1 : je dis bonjour de 1.1
UnThread T-1.2 : je dis bonjour de 1.2
UnThread T-2.1 : je dis bonjour de 2.1
UnThread T-2.2 : je dis bonjour de 2.2
UnThread T-3.1 : je dis bonjour de 3.1
Qui est où?
UnThread T-3.2 : je dis bonjour de 3.2
java.lang.ThreadGroup[name=groupe 1,maxpri=10]
  Thread[T-1.1,10,groupe 1]
  Thread[T-1.2,10,groupe 1]
java.lang.ThreadGroup[name=groupe 2,maxpri=1]
  Thread[T-2.1,1,groupe 2]
  Thread[T-2.2,1,groupe 2]
java.lang.ThreadGroup[name=groupe 3,maxpri=10]
  Thread[T-3.1,10,groupe 3]
  Thread[T-3.2,10,groupe 3]
Je n'ai plus rien à faire...
UnThread T-3.2 : bye bye
UnThread T-1.1 : bye bye
UnThread T-2.1 : bye bye
UnThread T-2.2 : bye bye
UnThread T-3.1 : bye bye
UnThread T-1.2 : bye bye
```

Hélas ce n'est pas
clair du tout...



Observation

Plus prioritaires = valeur maximum

Discussion sur les priorités

6

Une thread dispose de sa priorité

- Affectation réelle = $\min(\text{priorité}, \text{priorité max du groupe})$
- Par défaut...
 - ▶ Un Thread reprend la priorité de la thread qui la crée

L'impact des priorités sur l'ordonnancement?

- Pas évident (mon expérience)
- Lu sur le sujet
 - ▶ Dépendant de l'implémentation
- À utiliser avec précautions



Discussion sur les groupes de threads

7

Regroupements logiques possibles

- Facilité d'analyse
- Possibilité d'affecter des priorités différentes
 - ▶ Modulo une situation où cela joue

Relations possible avec des pool de threads

- Association possible à un groupe de thread...
- Grace à l'implémentation de l'interface ThreadFactory
 - ▶ Exemple montré plus loin (serveur multithreadé)